L1-2 讲义：大语言模型基础

课程: MBA 大模型智能体课程
课时: 第 1-2 讲（90 分钟）
适用: 讲师备课 / 学生预习与复习
对应 Slides: `slides-marp/day1-lesson1-2-llm-basics.md`

本讲核心问题

在开始之前，请带着这些问题阅读：

1. **什么是大语言模型？它和传统软件有什么根本区别？**
2. **Token 是什么？为什么它决定了你的 AI 成本？**
3. **Prompt 怎么写？如何让模型更好地理解你的意图？**
4. **什么是上下文工程？为什么说它是 Agent 时代的核心能力？**

Part 0：环境配置（15 分钟）

为什么先配置？

"工欲善其事，必先利其器。"这节课有大量动手实践环节，如果工具没配置好，你会一直卡在技术问题上，而不是学习 AI 本身。

现在花 15 分钟把环境搞定，后面 90 分钟才能真正学到东西。

你需要配置的三件事

序号	内容	预计时间
1	大模型服务账号 + API Key	5 分钟
2	OpenClaw 安装与配置	5 分钟
3	验证环境可用	5 分钟

Step 1：大模型服务账号

你至少需要注册一个大模型平台的账号。国内平台注册简单、有免费额度，建议优先选择。

国内平台（推荐先注册）：

平台	注册入口	特点
Kimi	kimi.moonshot.cn	中文长文本强，有 Coding Plan 订阅
DeepSeek	platform.deepseek.com	性价比极高，R1 推理模型
智谱清言	open.bigmodel.cn	清华系，GLM 模型

国际平台：

平台	注册入口	特点
OpenAI	platform.openai.com	GPT 系列，生态最完善
Anthropic	console.anthropic.com	Claude 系列，长文本强

平台	注册入口	特点
Google AI	aistudio.google.com	Gemini 系列，多模态

获取 API Key 的步骤（以 Kimi 为例）：

1. 访问 platform.moonshot.cn
2. 注册/登录账号
3. 进入「API 管理」
4. 点击「创建 API Key」
5. 复制保存（只显示一次！）

☐ **安全提示**：API Key 等同于密码，不要分享或提交到代码仓库。

Step 2：OpenClaw 安装与配置

OpenClaw 是一个 AI Agent 框架，可以让你通过飞书、Telegram 等渠道与 AI 对话，并执行复杂任务。

官方资源：

资源	链接
官网	openclaw.ai
完整教程	claw101.com/en ☐ 推荐
文档	docs.openclaw.ai

安装命令：

```
# macOS / Linux
curl -fsSL https://openclaw.ai/install | sh

# Windows PowerShell
irm https://openclaw.ai/install.ps1 | iex

# 或者用 npm
npm install -g openclaw
```

配置向导：

```
# 启动交互式配置
openclaw configure

# 验证安装
openclaw --version
```

Step 3：验证环境

```
# 发送测试消息
openclaw chat " 你好，请介绍一下你自己"
```

如果看到 AI 的回复，恭喜，环境配置完成！

Part 1：什么是大语言模型?

通俗理解

想象一个读过人类所有书籍、文章、代码的"超级读者"。它不是真正理解内容，而是学会了**语言的统计规律**——给定前文，预测下一个最可能出现的词。

输入："今天天气真"
模型预测："好"（概率85%）/ "糟"（概率10%）/ "热"（概率5%）

这个简单的"预测下一个词"任务，在足够大的数据和模型规模下，涌现出了惊人的能力：推理、创作、编程、翻译……

AI 的 iPhone 时刻

2022 年 11 月 30 日，OpenAI 发布 ChatGPT。这是生成式 AI 进入大众认知的标志性事件。

产品	达到 1 亿用户时长
电话	75 年
互联网	7 年
TikTok	9 个月
ChatGPT	**2 个月**

这不仅是一个技术里程碑，更是商业范式的转变。MBA 学生需要理解：**AI 正在从"试点工具"变成"核心生产力"**。

企业采用的拐点（2024-2026）

过去两年，企业对 AI 的态度发生了根本变化：

- 从"试点 AI 工具"转向"**AI 原生流程重构**"
- 从"单点问答"转向"**多模型 + 工作流 + 评估体系**"
- 从"炫技 Demo"转向"**ROI/合规/可审计**"

MBA 视角：竞争优势开始由"会不会用 AI"变成"能否系统化落地 AI"。

技术演进时间线

年份	里程碑	意义
2017	Transformer 论文	注意力机制革命，告别 RNN
2018	GPT-1, BERT	预训练 + 微调范式
2020	GPT-3 (175B)	规模涌现，Few-shot 学习
2022	ChatGPT	对话式 AI 走向大众
2023	GPT-4	多模态，推理能力飞跃
2024	Claude 3/GPT-4o	长上下文，原生多模态
2025	o1/o3/DeepSeek-R1	推理模型，思维链原生
2026	Claude 4.5/GPT-5.3	Agent 原生，工具调用优化

Transformer 架构（MBA 版）

你不需要理解数学细节，只需知道 Transformer 的三个关键创新：

1. **注意力机制 (Attention)**：模型可以"关注"输入中任意位置的信息，不受距离限制
2. **并行计算**：可以同时处理整个序列，训练速度大幅提升
3. **可扩展性**：架构简洁，容易通过增加参数获得更强能力

推荐阅读：Jay Alammar 的图解 Transformer是最好的可视化教程。

> **MBA 视角**：Transformer 的发明者大多已离开 Google 创业。这项技术创造了万亿美元的新市场，但发明者的直接回报有限——这是一个关于创新价值分配的经典案例。

2026 年模型格局

你必须认识的五类模型：

模型	厂商	核心优势	典型场景
GPT-5.3 Pro	OpenAI	综合能力强、工具生态成熟	复杂通用任务
Claude 4.5	Anthropic	稳定、长文档处理优秀	法务/研究/报告
DeepSeek V3	DeepSeek	高性价比通用模型	大规模业务调用
DeepSeek R1	DeepSeek	深度推理能力	分析推导任务
Kimi K2.5	月之暗面	中文长文本 + 知识处理	读材料、做综述

Part 2：Token 经济学

什么是 Token?

Token 是 LLM 处理文本的基本单位，不是字符，也不是单词，而是介于两者之间的"子词"。

```
"Hello, world!" → ["Hello", ",", " world", "!"]  (4 tokens)
"人工智能" → ["人工", "智能"]  (2 tokens)
```

商业直觉：Token 是 AI 的"计费单位"，就像电话按分钟计费、打车按公里计费一样，LLM 按 token 计费。

为什么 MBA 必须懂 Token?

场景	Token 数	GPT-4 成本	商业启示
一封邮件	~200	$0.006	几乎免费
一份 10 页报告	~3,000	$0.09	仍然很便宜
1000 封客服对话	~500,000	$15	比雇人便宜 100 倍
分析一本书	~100,000	$3	人工需要一周

成本计算公式

`总成本 = 输入Token × 输入单价 + 输出Token × 输出单价`

2026 年 API 定价参考（每百万 Token，USD）：

模型	输入价格	输出价格
GPT-5.3 Pro	$10	$30
Claude Opus	$5	$25
DeepSeek V3	$0.5	$2

> **决策启示**：简单任务用便宜模型，复杂任务用强模型。

Part 2.5：LLM 是如何训练出来的？（Post-Training 方法）

三阶段训练流水线

大模型的诞生需要经过三个阶段：

```
Stage 1: Pretraining（预训练）
    ↓ 海量文本，学习语言和知识
Stage 2: SFT（监督微调）
    ↓ 高质量对话数据，学习对话格式
Stage 3: Post-Training（后训练对齐）
    ↓ RLHF/DPO/GRPO/RLAIF，对齐人类偏好
```

Post-Training 方法演进

时期	方法	代表模型	特点
2022-2023	RLHF (PPO)	ChatGPT, Claude 1-2	需要 Reward Model + PPO 训练
2024	DPO	Llama 3, Qwen 2	直接优化，无需 Reward Model
2025	GRPO	DeepSeek R1	可验证奖励，适合推理任务
2026	RLAIF	Claude 4, Kimi K2	用 AI 替代人工反馈

方法对比速查

方法	需要 RM	复杂度	最适合场景
RLHF	☐ 需要	高	通用对话对齐
DPO	☐ 不需要	中	快速迭代优化
GRPO	☐ 不需要	中	推理/数学/代码
RLAIF	☐ 不需要	中	大规模低成本训练

MBA 视角：商业含义

1. **门槛在下降**：从 RLHF 到 DPO/GRPO，对齐训练成本大幅降低
2. **小团队也能做**：DeepSeek 用 GRPO 训练出顶级推理模型
3. **数据依赖减少**：RLAIF 用 AI 生成反馈，不再依赖大量人工标注
4. **垂直领域机会**：企业可以在特定领域做 SFT+DPO 微调

   创业启示：2024 年之后，中小团队用 DPO/GRPO 也能训练出有竞争力的垂直模型。

Part 3：上下文窗口与注意力预算

什么是上下文窗口？

上下文窗口（Context Window）是模型单次能"看见"的最大 token 数。

商业直觉：Context Window 就是 AI 的"工作记忆"容量。想象一位助理：- 有的助理只能同时处理 3 页文件（小窗口）- 有的助理能同时处理 300 页文件（大窗口）

2026 年主流模型上下文窗口

模型	上下文窗口	等于多少页文档
GPT-4 Turbo	128K tokens	~200 页
Claude 4	200K tokens	~300 页
Gemini 2	2M tokens	~3000 页
Kimi K2.5	2M tokens	~3000 页

注意力预算（Attention Budget）

这是理解 AI 行为的关键概念。

核心原理：LLM 就像人类一样，工作记忆容量有限。每增加一个 token，都会消耗一部分"注意力预算"。

Anthropic 在 2025 年的研究中指出：

> "Like humans, who have limited working memory capacity, LLMs have an 'attention budget'that they draw on when parsing large volumes of context. Every new token introduced depletes this budget by some amount."

用 CEO 的日程来理解：

想象你是 CEO，今天有 100 件事需要决策：- 如果只有 5 件重要的事 → 每件事可以深度思考 20 分钟 - 如果有 100 件混杂的事 → 每件事只能分到 1 分钟，决策质量下降

LLM 也是如此：**上下文越长，每个 token 获得的"注意力"就越少**。

上下文腐烂（Context Rot）

随着上下文变长，模型准确回忆信息的能力会下降。这不是 bug，而是架构特性。

```
上下文长度          信息召回准确率

10K tokens   →   95%
50K tokens   →   85%
100K tokens  →   70%
200K tokens  →   55%
```

MBA 启示：不要以为"窗口大就能塞更多"，而是要思考"如何让有限的注意力聚焦在最重要的信息上"。

KV-Cache：为什么它对成本至关重要?

用会议记录来理解 KV-Cache

想象你是一位 CEO，每周一都要开同样的例会：

方式 A：无缓存（低效）

```
第1周：花30分钟回顾公司背景、战略方向、部门职责...然后开会
第2周：花30分钟回顾公司背景、战略方向、部门职责...然后开会
第3周：花30分钟回顾公司背景、战略方向、部门职责...然后开会
```

每次都从头理解，效率极低。

方式 B：有缓存（高效）

```
第1周：花30分钟理解背景，把理解"缓存"到笔记本
第2周：翻开笔记本，5分钟回顾，直接开会
第3周：翻开笔记本，5分钟回顾，直接开会
```

第一次建立理解后，后续可以复用。

KV-Cache 就是 LLM 的"笔记本"——它保存了模型对之前内容的"理解"，避免重复计算。

商业影响：

场景	Claude Sonnet 成本	差距
缓存命中	$0.30/百万 token	基准
缓存未命中	$3.00/百万 token	**10 倍！**

产品经理必须知道的规则：

☐ **保持 System Prompt 前缀稳定**—同样的开头可以复用缓存 ☐ **Context 只追加，不修改历史**—修改会破坏缓存 ☐ **不要在开头放时间戳**—每次都变会导致缓存失效

Manus 团队的实战经验：

> "Mask, not Remove"（遮盖，而非删除）

当需要"忘记"某些历史信息时，不要删除消息（会破坏缓存），而是在 prompt 中指示模型"忽略之前关于 X 的讨论"。这样既能调整行为，又能保持缓存命中。

Part 4：Prompt 工程基础

> **学习路径**：先掌握单次调用的 Prompt（本节），再学习多轮 Agent 的上下文工程（Part 6）。

Prompt 工程 vs 上下文工程

在深入之前，先理解这两个概念的关系：

维度	Prompt 工程	上下文工程
定义	写好一次性的指令	管理整个信息流
适用场景	单次问答、简单任务	多轮对话、Agent 系统
核心问题	"这句话怎么写？"	"应该放什么信息进去？"
关系	基础	进阶

类比：Prompt 工程就像写好一封邮件；上下文工程就像管理整个邮件往来记录、相关文档、历史决策，确保对方能做出正确判断。

本节先讲 **Prompt 工程**（单次调用的基础），Part 6 再讲**上下文工程**（Agent 系统的进阶）。

Prompt 五要素

要素	说明	示例
角色 (Role)	你是谁	"你是一位有 15 年经验的投资分析师"
任务 (Task)	要完成什么	"分析特斯拉 2025Q4 财报的三个关键风险"
格式 (Format)	结果长什么样	"以表格形式输出"
约束 (Constraints)	边界条件	"字数 300-500 字，必须引用数据来源"
示例 (Examples)	参考答案风格	给 1-3 个输入输出示例

示例对比

☐ **糟糕的 Prompt**：

帮我写个行业分析

☐ **优秀的 Prompt：**

角色：你是消费行业战略顾问（12年经验）。

任务：评估A品牌进入东南亚市场可行性。

格式：
1）五维评分表（市场/竞争/渠道/合规/财务，1-10分）
2）90天行动计划（按周）
3）关键风险与预案

约束：
- 不确定信息必须标注
- 每条建议需给依据
- 字数1000-1500字

Few-shot：用例子说话

Anthropic 强调：

> "For an LLM, examples are the 'pictures'worth a thousand words."

与其写一堆规则，不如给几个好例子：

任务：判断客户评价的情感倾向

示例1：
评价："物流很快，产品质量不错"
情感：正面

示例2：
评价："等了一周才到，包装还破损"
情感：负面

请判断：
评价："东西一般般，价格倒是便宜"
情感：

Chain-of-Thought (CoT)

要求模型显示中间推理步骤：

直接问："这个创业项目值得投资吗？"

CoT问法："请一步步分析这个创业项目：
第一步：市场规模和增长潜力
第二步：竞争格局和壁垒
第三步：团队背景和执行力
第四步：商业模式和盈利路径
第五步：综合以上分析，给出投资建议"

为什么有效：强制模型展示推理过程，减少跳步导致的错误。

System Prompt 的"最优高度"

Anthropic 提出了一个重要概念：prompt 的"高度"(altitude)。

太具体（脆弱）　　太模糊（无效）

```
"如果A问X回答Y                    "做一个有帮助的助手"
 如果B问W回答Z..."

                        刚好（灵活且有指导）

                      "你是客服助手。核心原则：
                      1. 先确认理解再回答
                      2. 退款请求先查订单状态
                      3. 无法解决时转人工"
```

原则：具体到足以指导行为，但灵活到可以处理边缘情况。

工具设计原则

> "One of the most common failure modes we see is **bloated tool sets** that cover too much functionality or lead to ambiguous decision points."
>
> —Anthropic

坏设计 vs 好设计：

```
  20个相似工具                  最小工具集

search_products()             search_products(
find_products()                 query,
query_products()                filters
get_product_list()            )
lookup_products()
```

判断标准：如果人类工程师都不确定用哪个，AI 也做不到。

Part 5：从单次调用到 Agent

为什么需要新的思维模式?

当你从"单次问答"进入"Agent 系统"时，思维模式需要根本性转变：

维度	单次调用	Agent 系统
交互	一问一答	多轮循环
持续时间	几秒	几分钟到几小时
上下文	静态	动态累积
核心挑战	写好 Prompt	管理上下文状态

Agent 的基本循环

```
用户请求


          Agent 循环

      1. 理解当前状态
      2. 选择下一步动作
      3. 执行动作（调用工具）
```

```
4. 观察结果
5. 判断是否完成

    ↓ 未完成则继续
```

最终结果

上下文的累积问题

每一轮循环都会增加上下文：

```
第1轮：用户请求 + Agent思考 + 工具调用 + 结果
第2轮：+ Agent思考 + 工具调用 + 结果
第3轮：+ Agent思考 + 工具调用 + 结果
...
第50轮：上下文已经膨胀到100K+ tokens
```

这就是为什么 Agent 开发需要"上下文工程"——管理这个不断增长的信息流。

Part 6：上下文工程（Context Engineering）

定位：这是 Agent 时代的核心能力，也是本讲的重点进阶内容。

什么是上下文工程?

Andrej Karpathy 将 LLM 比作一种新的操作系统：LLM 是 CPU，上下文窗口就是 RAM。就像操作系统管理 RAM 中的内容一样，**上下文工程就是管理 LLM"工作记忆"的艺术和科学**。

> "Context engineering is the delicate art and science of filling the context window with just the right information for the next step."
>
> —Andrej Karpathy

为什么这是 Agent 开发的核心?

Cognition（Devin 的开发者）直言：

> "Context engineering is effectively the **#1 job** of engineers building AI agents."

Anthropic 也强调：

> "Agents often engage in conversations spanning **hundreds of turns**, requiring careful context management strategies."

当 Agent 执行长时间任务时，上下文会不断累积，导致三个问题：

问题	表现	后果
Context Poisoning	幻觉进入上下文	错误被放大
Context Distraction	无关信息太多	模型被带偏
Context Confusion	信息相互矛盾	决策混乱

上下文的六个层次

现代 Agent 的上下文不只是"一个 prompt"，而是多层结构：

```
                    上下文的六层结构


  Layer 1: System Rules（系统规则）
     行为准则、安全约束、角色定义

  Layer 2: Memory（记忆）
     长期偏好、历史决策、学习到的经验

  Layer 3: Retrieved Docs（检索文档）
     RAG获取的相关知识

  Layer 4: Tool Schemas（工具定义）
     可用工具的接口描述

  Layer 5: Conversation（对话历史）
     最近的交互记录

  Layer 6: Current Task（当前任务）
     用户最新的请求
```

关键洞察：每一层都要"小而精准"（small and on-purpose）。

四大上下文管理策略

LangChain 总结了四种核心策略：**Write**、**Select**、**Compress**、**Isolate**。

策略 1：Write（写入外部存储） 把信息写到上下文窗口之外，需要时再读取。

Scratchpad（草稿本）

Agent 在执行任务时记笔记，类似人类做项目时写备忘录：

```
# task_notes.md

## 目标
分析竞争对手定价策略

## 已发现
- 竞品 A：订阅制，$29/月
- 竞品 B：按使用量计费
- 竞品 C：免费增值模式

## 下一步
- 对比各模式的客户留存率
```

为什么有效：即使上下文被截断，关键信息仍然保存在文件中。

Manus 的实践：

> "We treat the file system as the ultimate context: unlimited in size, persistent by nature, and directly operable by the agent itself."
>
> —Manus 团队

策略 2：Select（精准选择） 只把相关信息拉入上下文，而非"全部塞进去"。

代码 Agent 的做法：

Agent	配置文件	作用
Claude Code	CLAUDE.md	项目规则、偏好设置
Cursor	.cursorrules	代码风格、框架约定
Windsurf	rules 文件	团队规范

这些是"程序性记忆"——总是被加载的核心指令。

知识选择的挑战：

Windsurf 工程师 Varun 指出：

> "Indexing code ≠ context retrieval. Embedding search becomes unreliable as codebase grows. We rely on grep, knowledge graph, and re-ranking."

最佳实践：结合多种检索方式，而非单一依赖向量搜索。

策略 3：Compress（压缩） 只保留完成任务必需的 token。

上下文总结：

当对话超过一定长度时，用模型总结历史：

```
原始对话：50,000 tokens

        模型总结
压缩后：5,000 tokens

    包含：关键决策、未完成任务、重要发现
    丢弃：冗余的工具输出、已解决的讨论
```

Claude Code 在上下文达到 95% 时自动执行"auto-compact"。

工具输出压缩：

对于 token 密集型工具（如网页搜索），只保留摘要：

```
搜索工具返回：10,000 tokens的原始内容

        后处理
保留：500 tokens的结构化摘要
```

策略 4：Isolate（隔离） 用子 Agent 隔离上下文，防止污染。

```
主Agent（轻量上下文）

        子Agent A：深度搜索
            独立的干净上下文
            消耗30,000 tokens
            返回2,000 tokens摘要
```

```
子Agent B: 代码分析
    独立的干净上下文
    消耗20,000 tokens
    返回1,500 tokens摘要
```

Anthropic 的多 Agent 研究系统就采用这种架构：

> "Each subagent might consume tens of thousands of tokens, but only returns a refined summary of 1,000-2,000 tokens."

Manus 的六条实战经验

Manus 团队在构建 Agent 过程中总结了六条关键经验：

1. 围绕 KV-Cache 设计

> "If I had to choose just one metric, the **KV-cache hit rate** is the single most important metric for a production-stage AI agent."

具体做法：

做法	原因
保持 prompt 前缀稳定	单 token 变化会使后续缓存失效
上下文只追加不修改	修改历史会破坏缓存
不在开头放时间戳	每次都变会导致缓存失效
确保 JSON 序列化确定性	键顺序不一致会破坏缓存

2. Mask, Don't Remove（遮盖而非删除） 当工具太多时，不要动态删除工具定义（会破坏缓存），而是用 **token masking** 限制可选范围。

```
工具定义（始终存在）：
- browser_search
- browser_click
- shell_run
- file_read

当前状态限制：
只允许选择 browser_* 开头的工具
```

技巧：设计工具名称时使用一致的前缀（browser_、shell_、file_），便于分组限制。

3. 用文件系统作为上下文 把文件系统当作"无限大的外部上下文"：

- **压缩是可恢复的**：网页内容可以丢弃，因为 URL 保留着
- **文档内容可省略**：只要文件路径还在，随时可以重新读取

4. 通过"复述"操纵注意力 Manus 在处理复杂任务时会创建 todo.md，并不断更新：

```
# todo.md

- [x] 收集竞品数据
- [x] 分析定价策略
- [ ] 撰写对比报告 ← 当前
- [ ] 生成建议
```

为什么有效：不断复述目标，把任务推到上下文末尾，保持模型"不走神"。

5. 保留错误记录

> "One of the most effective ways to improve agent behavior is deceptively simple: **leave the wrong turns in the context.**"

当 Agent 看到失败的尝试和错误信息时，会隐式更新自己的判断，避免重复同样的错误。

反直觉但有效：不要急于清理错误，让模型从错误中学习。

6. 避免被 Few-shot 带偏 如果上下文中有太多相似的 action-observation 对，模型会倾向于模仿这个模式，即使不再适用。

解决方案：引入结构化变异——不同的序列化模板、措辞变化、顺序调整。

System Prompt 的最优高度

Anthropic 提出的"Goldilocks Zone"概念：

```
太具体（脆弱）                    太模糊（无效）

"如果A问X回答Y                    "做一个有帮助的助手"
 如果B问W回答Z..."

                        刚好（灵活且有指导）

                       "你是客服助手。核心原则：
                       1. 先确认理解再回答
                       2. 退款请求先查订单状态
                       3. 无法解决时转人工"
```

工具设计原则

> "One of the most common failure modes we see is **bloated tool sets** that cover too much functionality or lead to ambiguous decision points."
>
> —Anthropic

坏设计 vs 好设计：

```
20个相似工具                  最小工具集

search_products()             search_products(
find_products()                 query,
query_products()                filters
get_product_list()            )
lookup_products()
```

判断标准：如果人类工程师都不确定用哪个，AI 也做不到。

Part 7：Temperature 与推理模型

Temperature：随机性旋钮

Temperature	结果特征	适用场景
0.0-0.3	稳定、保守	数据抽取、财务分析
0.4-0.7	平衡	商业分析、方案草拟
0.8-1.2	发散、创意	文案、命名、点子

Temperature	结果特征	适用场景

两种模型工作范式

类型	特点	代表
System 1（快）	模式匹配、响应迅速	GPT-4o、Claude Sonnet
System 2（慢）	分步推理、自检纠错	o3、DeepSeek-R1

何时使用推理模型：多约束决策、数学证明、代码调试、因果分析。

Part 8：安全与风险

Prompt 注入

攻击者可能通过恶意输入让模型忽略原始指令。

防护策略：1. 输入层：清洗与分类 2. 指令层：明确优先级 3. 执行层：工具权限最小化 4. 输出层：结果校验与审计

幻觉问题

高风险场景：法务、医疗、金融、学术引用

降低幻觉：- 要求给出处/证据 - 区分"事实/推断/不确定"- 引入 RAG 检索 - 关键结论做人审

> **规则**：高价值决策必须"人机协同闭环"。

课堂实操

分组练习（15 分钟）

每组选择一个场景，设计上下文策略：

1. **客服 Agent**：处理退款请求
2. **研究 Agent**：竞争对手分析
3. **写作 Agent**：周报生成

需要回答：- System Prompt 的"高度"如何把握？- 需要哪些工具？有没有功能重叠？- 采用什么检索策略（预计算/JIT/混合）？- 如何处理长对话（压缩/笔记/子 Agent）？

本讲总结

核心概念升级

旧概念	新概念	区别
Prompt 工程	**上下文工程**	从"写好一句话"到"管理整个信息流"
上下文窗口	**注意力预算**	不是"能装多少"，而是"注意力如何分配"
越大越好	**最小高信号集**	用最少的 token 达到最好的效果

MBA 管理者必知

1. **Token = 成本单位**：理解定价才能做 ROI 分析
2. **KV-Cache = 效率杠杆**：设计得当可省 10 倍成本
3. **上下文工程 = Agent 的核心竞争力**：不是写好 Prompt 就够了

Anthropic 的总结

> "Even as capabilities scale, treating context as a precious, finite resource will remain central to building reliable, effective agents."

把上下文当作宝贵的有限资源——这是构建可靠 Agent 的核心思想。

延伸阅读

必读

- Anthropic: Effective Context Engineering for AI Agents (2025.9.29) □
- Anthropic: Building Effective Agents

官方文档

- Anthropic Prompt Library
- OpenAI Prompting Guide

视频

- Andrej Karpathy: Intro to LLMs
- Andrej Karpathy: Deep Dive into LLMs

本讲义最后更新：2026-02-28